

Answer 1

No changes would have to be made since the DBMS provides independence between the application layer and the physical layer.

Answer 2

A relation is a set of unordered columns and rows. Therefore, the relations are identical since they have the same attributes and contain identical information.

Answer 3

$$\pi_{T.cname}(\sigma_{T.credits > S.credits}(\sigma_{S.dept = \text{"CSE"}}(\rho_T(Course) \times \rho_S(Course))))$$

A better expression would be

$$\pi_{T.cname}(\sigma_{T.credits > S.credits}(\pi_{T.cname, T.credits}(\rho_T(Course)) \times \pi_{S.credits}(\sigma_{S.dept = \text{"CSE"}}(\rho_S(Course)))))$$

This expression reduces **Course(S)** to only those values corresponding to credits of courses in CSE, which should be a small set. It also eliminates the unneeded attributes from **Course(T)**. The cartesian product is therefore performed on the smallest amount of data.

Answer 4

1. Yes, there is a relational algebra equivalent for LOJ :

$$R \text{ LOJ } S = \sigma_{\theta}(R \times S) \cup ((R - \pi_R(\sigma_{\theta}(R \times S))) \times \{(null, null, \dots, null)\})$$

where the constant null relation is on the schema $S - R$. In the above, θ is the join predicate.

2. LOJ is not associative. Consider schema $R1(A, B)$; $R2(A, C)$; $R3(A, D)$ with contents $R1 = (x, 1)$, $R2 = (y, 2)$, $R3 = (x, 3)$, then

$$R1 \text{ LOJ } (R2 \text{ LOJ } R3) = (x, 1, null, null)$$

while

$$(R1 \text{ LOJ } R2) \text{ LOJ } R3 = (x, 1, null, 3)$$

3. LOJ is not commutative since $-$ (minus operator) itself is not commutative.
4. Since every tuple of R has to appear in the output, the minimum cardinality of $R \text{ LOJ } S$ is $|R|$.

Answer 5

Overall Approach:

```
SELECT cname FROM COMPANY
WHERE (type = 'private') AND
      cname NOT IN ((PARTIAL_DIST) UNION (NON_LOCAL or NON-3))
ORDER BY cname ASC
```

PARTIAL_DIST:

```
SELECT p.cname FROM PROJECT p
GROUP BY p.cname
HAVING ((COUNT (DISTINCT pstate) > 1) AND
        (COUNT (DISTINCT pstate) <> COUNT (pname)))
```

NON_LOCAL:

```
SELECT h.cname FROM HIRE h, PROJECT p, WORKER w
WHERE (h.cname = p.cname) AND (h.pname = p.pname)
      AND (h.EBno = w.EBno) AND ((p.pstate <> w.wstate) OR
```

NON-3:

```
((SELECT COUNT (*) FROM EXPERIENCE e
WHERE (w.EBno = e.EBno)) <> 3))
```

Full Answer:

```
SELECT cname FROM COMPANY
WHERE (type = 'private') AND cname NOT IN
      ((SELECT p.cname FROM PROJECT p
        GROUP BY p.cname
        HAVING ((COUNT (DISTINCT pstate) > 1) AND
                (COUNT (DISTINCT pstate) <> COUNT (pname))))
      UNION
      (SELECT h.cname FROM HIRE h, PROJECT p, WORKER w
        WHERE (h.cname = p.cname) AND (h.pname = p.pname) AND
              (h.EBno = w.EBno) AND ((p.pstate <> w.wstate) OR
              ((SELECT COUNT (*) FROM EXPERIENCE e
                WHERE (w.EBno = e.EBno)) <> 3))))
ORDER BY cname ASC
```

In addition, to eliminate companies that have one or more “workerless” projects, we can add:

```
AND cname NOT IN
      (SELECT p.cname FROM PROJECT p
       WHERE pname IN
             ((SELECT pname FROM PROJECT p)
              EXCEPT (SELECT pname FROM HIRE)))
```